

Decision Intelligence Topologies: Scaling Auditable Autonomy

0. Abstract

Disclaimer: This document describes system topologies and communication patterns for autonomous agents. It assumes the existence of the **Decision Intelligence Runtime (DIR)** to handle deterministic execution and the **Responsibility-Oriented Agents (ROA)** framework to define agent identity.

The transition from experimental AI scripts to production-grade decision systems requires a fundamental shift in how agents interact within an environment. A single architectural pattern cannot satisfy the conflicting requirements of complex, multi-perspective strategic reasoning, high-frequency tactical execution, and absolute compliance verification.

This whitepaper introduces **Decision Intelligence Topologies**, a pluralistic approach to agent orchestration. We define three distinct operational modes:

1. **Topology A: Event-Oriented Agent Mesh (EOAM)** — A decentralized pattern for "Organizational Choreography," where responsibility-bound agents collaborate through a reactive event substrate to form a "Digital Twin" of professional decision-making.
2. **Topology B: Sovereign Decision Stream (SDS)** — A linear, high-velocity pattern for "Atomic Execution," leveraging **Constrained Decoding** to achieve **Syntactically Bound by Design** operations at machine speeds.
3. **Topology C: Decision Ledger (DL+PCI)** — The pattern for "Formal Verification & Absolute Audit," focusing on the integrity of the intent artifact rather than the behavior of the agent.

By selecting the appropriate topology for the specific decision class, verified by **DecisionFlow IDs (DFID)** and governed by the **Decision Intelligence Runtime**, organizations can build systems that are scalable, auditable, and resilient to the non-deterministic nature of Large Language Models.

1. The Case for Architectural Pluralism

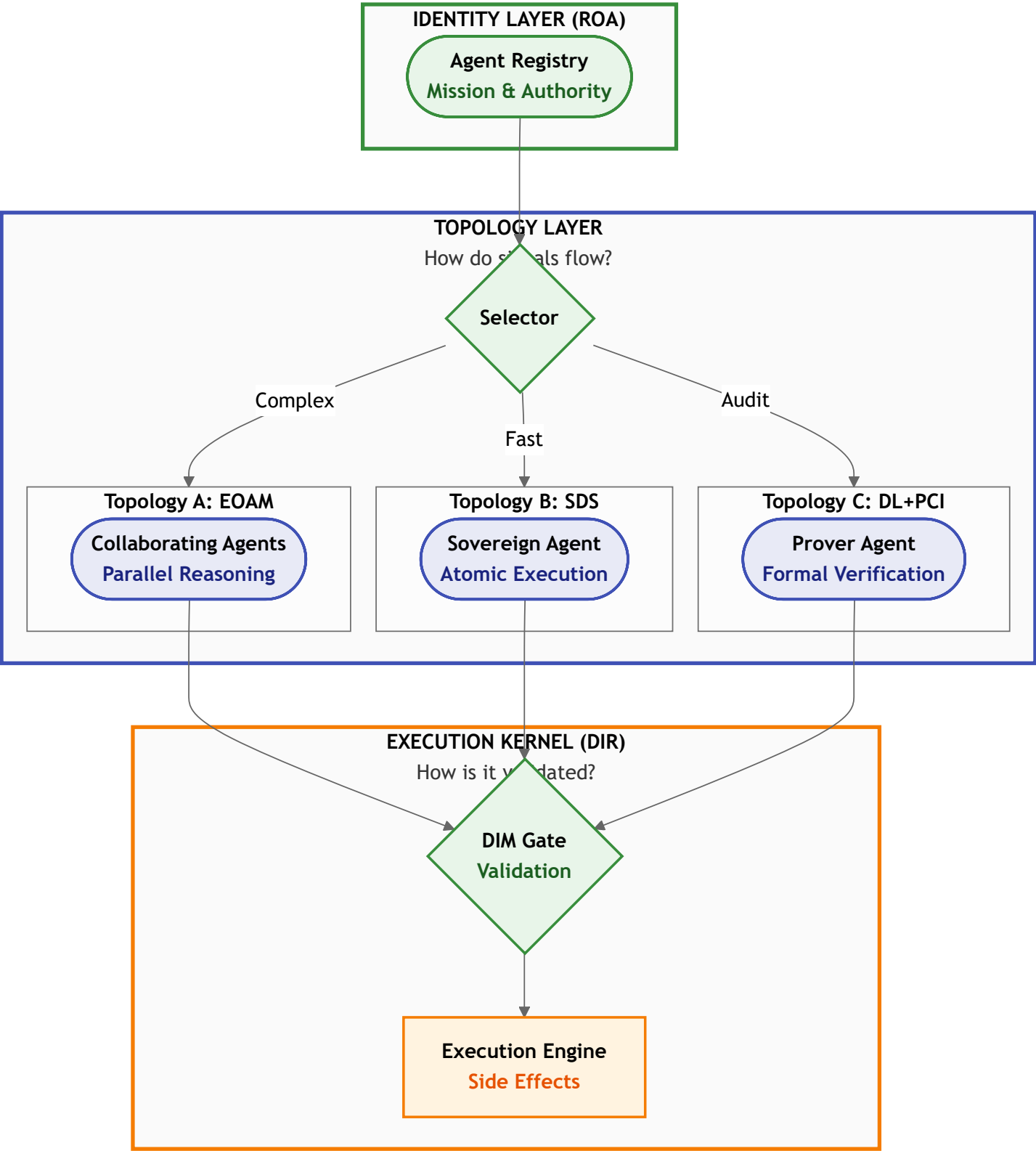
In the early phases of AI adoption, engineers often seek a "Universal Agent Architecture"—a single loop to rule them all. However, in production environments like the **Alvestor** financial system, this monolithic approach fails. The latency required to "think fast" (scalping a price inefficiency) is incompatible with the comprehensive context required to "think slow" (rebalancing a portfolio against geopolitical risk).

To solve this, we decouple the **Identity Layer** from the **Topology Layer**:

- **The Identity Layer (ROA):** Who acts? (Defined by Mission, Authority, and Responsibility Contracts).
- **The Execution Kernel (DIR):** How is it validated? (Defined by DFIDs, Hard Gates, and Idempotency).
- **The Topology (EOAM / SDS / DL):** How do signals flow?

This separation allows a single systems architecture to support deliberate mesh networks, hyper-fast singular streams, and formally verified ledgers side-by-side.

Architectural Pluralism: Decoupling Identity from Topology



2. Topology A: The Event-Oriented Agent Mesh (EOAM)

Primary Attribute: Organizational Choreography

Ideal For: Complex Strategy, Multi-Perspective Analysis, Resilience

EOAM is a decentralized architectural pattern where autonomous agents collaborate through a reactive event substrate. It defines a system that is "**Decentralized in activation, centralized in authority.**" EOAM trades the conceptual simplicity of linear orchestration for the power of **parallelism** and **resilience**.

2.1 Scope-Based Choreography

EOAM replaces the central "Manager" with **Scope-Based Choreography**. Agents do not wait for commands; they are reactive entities that monitor the event bus for **Observations** or **Triggers** that fall within their defined **Responsibility Contract**.

- **Autonomy through Subscription:** Agents subscribe to specific metadata—such as a ticker symbol or risk domain—defined in the **Agent Registry**.
- **Decentralized Intelligence:** When a new signal (e.g., a news event) enters the mesh, all relevant agents (Risk, Sentiment, Strategy) are activated in parallel.
- **Inversion of Control:** The Runtime does not "call" agents; it emits a context-rich event, and agents "claim" the responsibility to reason.

2.2 The Anatomy of a Mesh Event

Every event transmitted through the mesh adheres to a strict schema to facilitate causality tracking.

- **DecisionFlow ID (DFID):** The primary correlation identifier, functioning as an immutable trace header for the entire decision lifecycle. It allows for narrative reconstruction and deterministic idempotency.
- **ContextSnapshotID:** A unique hash representing the exact "frozen reality" that an agent utilized. This ensures that every **PolicyProposal** is linked to the version of the world the agent "saw," enabling **Just-In-Time (JIT) State Verification**.

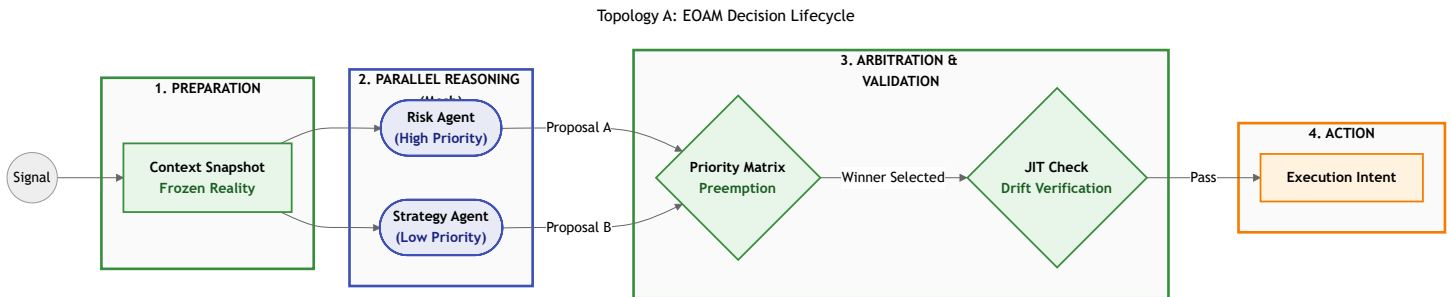
2.3 Semantic Routing & Economic Guardrails

In a decentralized mesh, mitigating noise and cost is critical.

- **Semantically Constrained Routing:** The underlying transport acts as a "dumb pipe," relying on rules from the **Agent Registry** to match topics to subscribers based on Least Privilege.
- **Wake-up Predicates (Signal Suppression):** To prevent "Token Burn" (waking up expensive LLMs for minor signals), the Runtime evaluates low-cost heuristic predicates (e.g., $\text{abs}(\text{price_delta}) > 0.5\%$) defined in the agent's manifest. If the predicate fails, the event is suppressed at the routing layer.
- **Economic Admission Control:** The Mesh implements a strict "Budget-to-Signal" filter. The system determines how many agents to activate based on signal volatility and available token budget, preventing "Thundering Herd" costs where low-value signals trigger expensive multi-agent cascades.

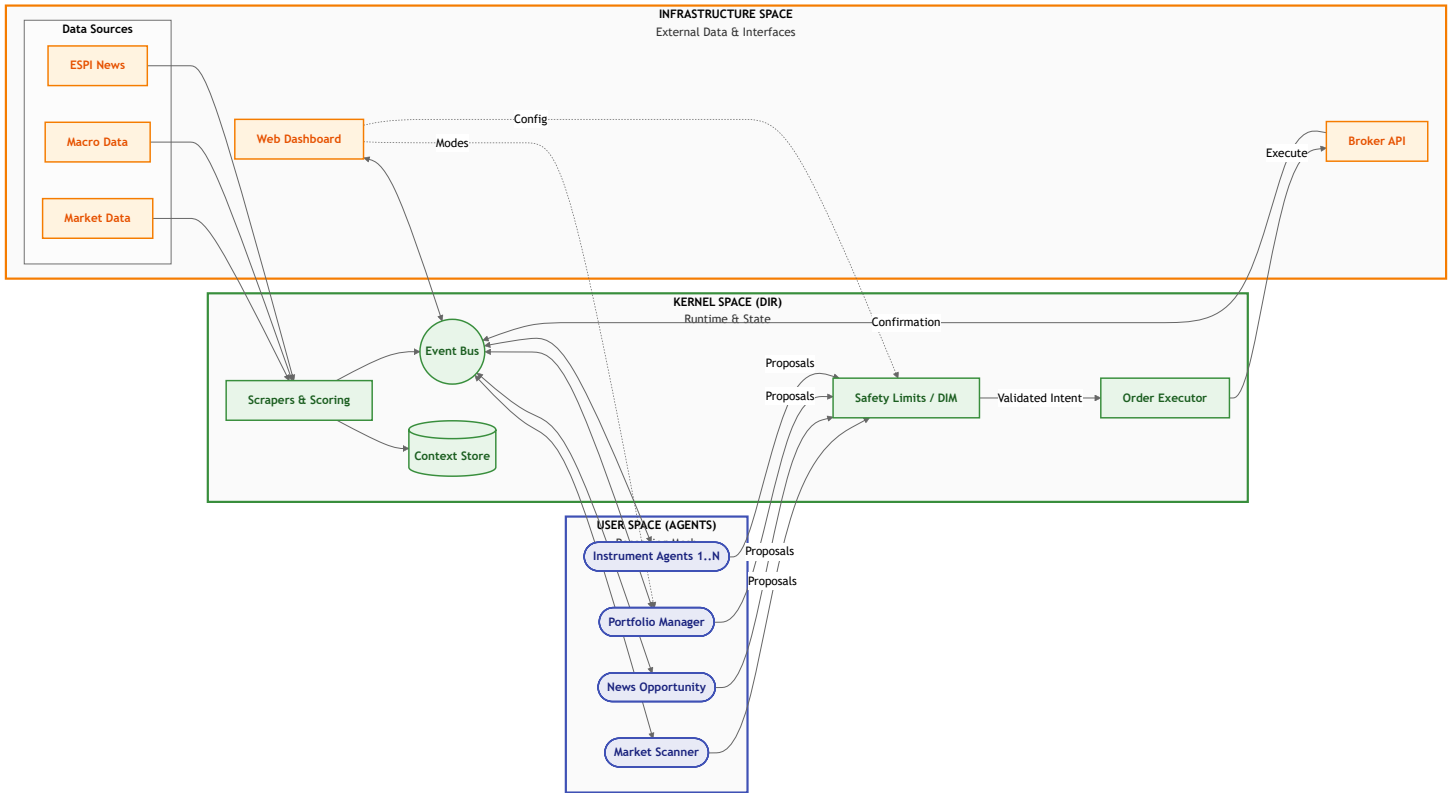
2.4 The Mesh Decision Lifecycle

1. **Observation:** A raw signal triggers a new **DecisionFlow**. The Runtime generates an authoritative **Context Snapshot** (cached to prevent "Thundering Herd" on the database).
2. **Distributed Parallel Reasoning:** Agents reason in parallel, producing **PolicyProposals**.
3. **Priority-Based Preemption Model:** The Runtime utilizes a priority-driven logic rather than a simple time window. It collects proposals and applies the **Agent Registry Priority Matrix** to select the winner. High-priority agents (e.g., Risk Monitors) can preempt or invalidate earlier strategic proposals, ensuring safety always overrides strategy.
4. **JIT Verification:** The Runtime compares the live state against the `ContextSnapshotID`. If the **Drift** exceeds the agent-defined **Drift Envelope** (a contract-level parameter stored in the Registry), the execution is rejected. This prevents actions based on stale data (e.g., execution against a price that has moved beyond the allowable slippage/latency threshold).



2.5 Holistic EOAM Architecture (Example Implementation)

The following diagram illustrates a complete "Event-Oriented Agent Mesh" implementation, derived from the **Alvestor** reference system. It demonstrates how Data Sources, Agents, and Execution components map to the DIR architectural spaces (Infrastructure, User, Kernel).



3. Topology B: The Sovereign Decision Stream (SDS)

Primary Attribute: Atomic Execution Velocity
Ideal For: High-Frequency Trading, Risk Stops, Tactical Reactions

While the **Event-Oriented Agent Mesh (EOAM)** excels at multi-perspective coordination and organizational "Digital Twins," it introduces significant latency and computational overhead due to its reliance on parallel orchestration and arbitration.

The **Sovereign Decision Stream (SDS)** is an alternative architectural pattern designed for high-frequency, low-latency, and cost-efficient decision-making. SDS treats the decision process as a "straight-line" function rather than a choreographic dialogue. It achieves structural safety not through post-generation validation (the "Audit" model), but through **Constrained Decoding**—embedding the deterministic rules of the **Decision Intelligence Runtime (DIR)** directly into the probabilistic sampling of the Large Language Model.

3.1 The SDS Philosophy: "Syntactically Bound by Design"

Mandatory Disclaimer: Constrained Decoding (Grammar-based sampling) ensures structural integrity and boundary adherence but **DOES NOT** guarantee semantic alignment with the ROA Mission. Final semantic and safety validation remains the exclusive responsibility of the DIR Decision Integrity Module (DIM).

In the SDS topology, the separation between User Space (Reasoning) and Kernel Space (Validation) remains architecturally absolute, but becomes executionally transparent. SDS replaces the "Thundering Herd" of parallel agents with a single, high-context **Decision Atom**.

3.1.1 Constrained Decoding (The Safety Straightjacket)

The cornerstone of SDS is the use of grammars (e.g., via libraries like *Outlines* or *Guidance*) to enforce the **Responsibility Contract** during inference.

- **Deterministic Sampling:** The LLM is physically unable to generate a token that violates the JSON schema or numerical bounds defined by the DIR.
- **Elimination of Syntax Errors:** Because the output is constrained by a state machine, the resulting **PolicyProposal** is guaranteed to be syntactically valid and semantically bounded before it even reaches the Runtime.

3.1.2 The "Decision Atom": Atomic Context

In SDS, there is no emergent context or asynchronous message passing. The **Context Compiler** assembles an "Atom"—a cryptographically signed package containing:

1. **The Mission** (from ROA).
2. **The Constraints** (from DIR Hard Gates).
3. **The Snapshot** (from the Context Store).

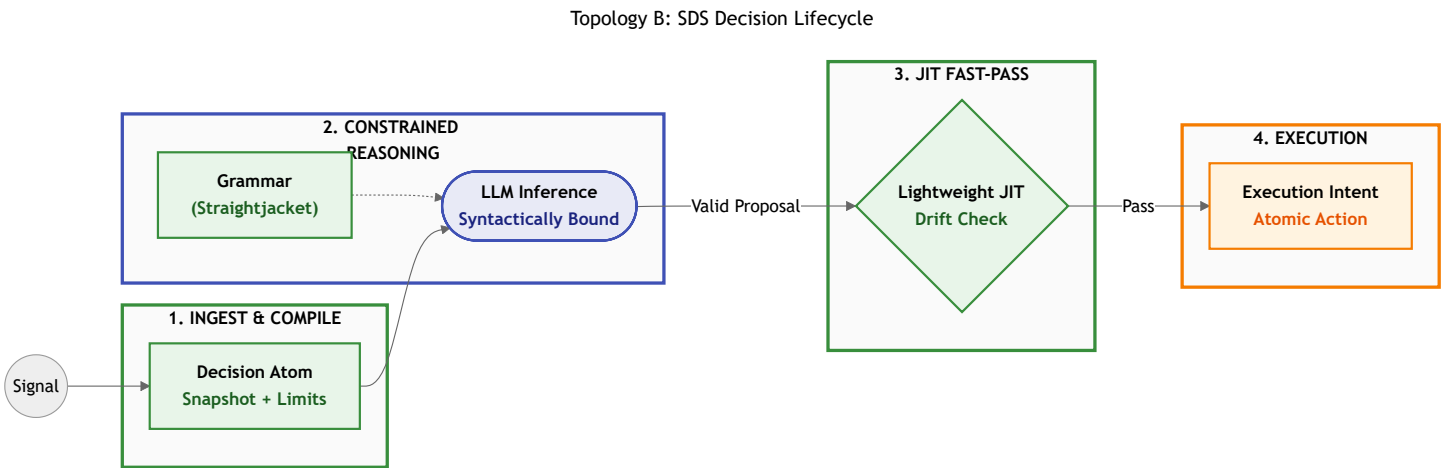
Critical Binding: The "Decision Atom" **MUST** include the `ContextSnapshotID` hash-binding. The DIR must verify this JIT to prevent execution against a drift-unverified state.

3.2 The SDS Decision Lifecycle

The SDS lifecycle collapses the multi-agent choreography of EOAM into a streamlined, four-stage pipeline.

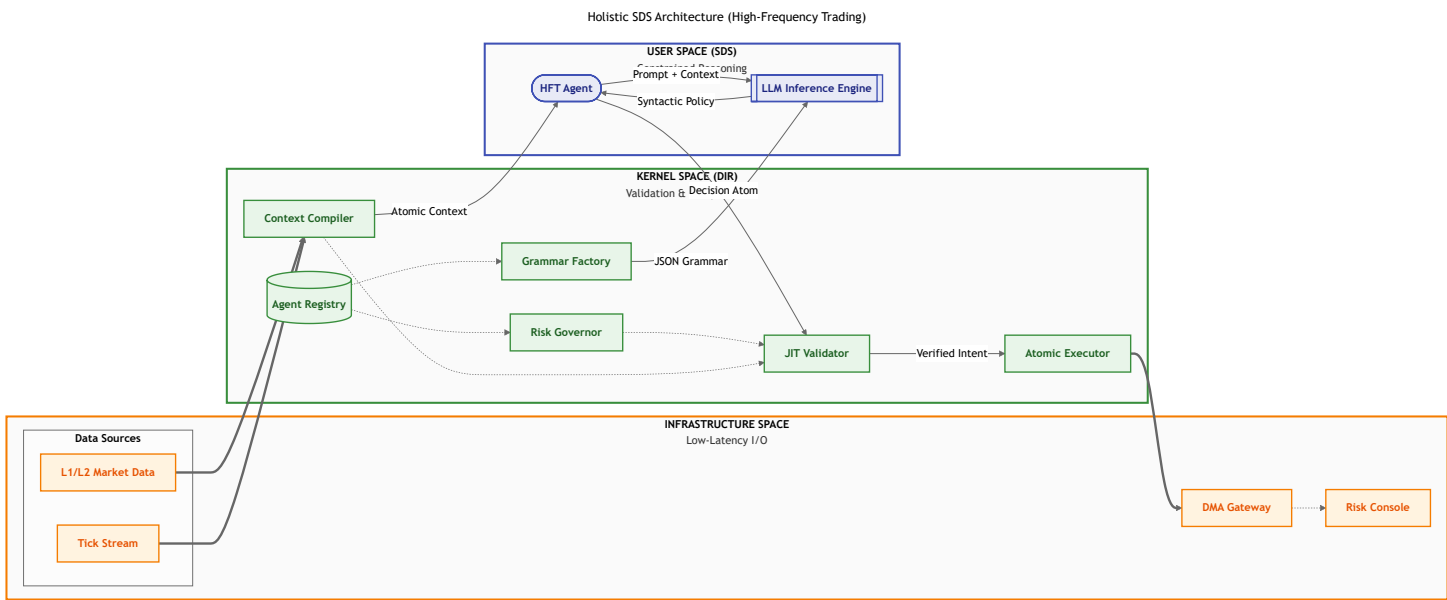
1. **Ingest & Compile:** A trigger event initializes a **DecisionFlow (DFID)**. The Runtime compiles the "Decision Atom," injecting authority limits directly into the prompt metadata.

- 2. **Constrained Reasoning (SDS Loop):** The agent performs its **Explain** phase. When transitioning to the **Policy** phase, the inference engine switches to a constrained mode, utilizing the DIR-provided grammar to ensure the proposal is "**Syntactically Bound by Design**".
- 3. **JIT Fast-Pass Validation:** Because the proposal was generated under constraint, the **Decision Integrity Module (DIM)** performs only a lightweight **Just-In-Time (JIT) State Verification** to check for environment drift since the snapshot.
- 4. **Execution:** The **PolicyProposal** is immediately transformed into an **Execution Intent**.



3.3 Holistic SDS Architecture (High-Performance Implementation)

The following diagram illustrates a complete **Sovereign Decision Stream (SDS)** architecture, designed for maximum throughput and minimal latency. It highlights the direct path from observation to execution, bypassing the complex event bus choreography of EOAM while maintaining strict safety via the "Straightjacket" grammar.



4. Topology C: The Decision Ledger (DL+PCI)

Primary Attribute: Formal Verification & Absolute Audit

Ideal For: Compliance-Heavy Operations, Inter-Organizational Settlements, High-Value Transfers

Topology C represents the ultimate stage of formal verification within the Decision Intelligence Runtime framework. It shifts the focus from "trusting the agent" (EOAM) or "constraining the agent" (SDS) to **"verifying the artifact."**

4.1 The Philosophy: Proof-Carrying Intents (PCI)

In this model, the system operates on a "Zero-Trust" basis regarding the agent's reasoning process. The **Decision Ledger** does not execute an intent because an agent "said so"; it executes it only if the intent carries an immutable proof of compliance.

This transitions the architecture from **Active Agents** to **Proof-Carrying Intents (PCI)**. The safety is a property of the data structure itself, not the process that generated it.

4.2 The Decision Ledger

The **Decision Ledger** is an append-only, immutable record that serves as the "Immutable Source of Truth" for the system.

- **Append-Only:** History cannot be rewritten. Every decision, whether successful or rejected, is permanently recorded.
- **Idempotency:** The Ledger ensures that replaying the same intent results in the same outcome (or a deterministic rejection if state has changed).
- **Zero-Trust Auditability:** Auditors do not need to inspect the LLM's "thoughts"; they only need to verify the cryptographic proofs stored in the Ledger.

4.3 The Proof Checker

In Topology C, the **Decision Integrity Module (DIM)** acts as a minimalist **Proof Checker**. It does not "think" or "reason." It simply validates the proofs attached to the intent.

- **Cryptographic Verification:** Verifies the signature of the intent (binding it to an ROA identity).
- **Logical Verification:** Checks that the intent includes valid evidence for all required rules (e.g., "RBAC Check Passed," "Limit Check Passed," "State Freshness Verified").

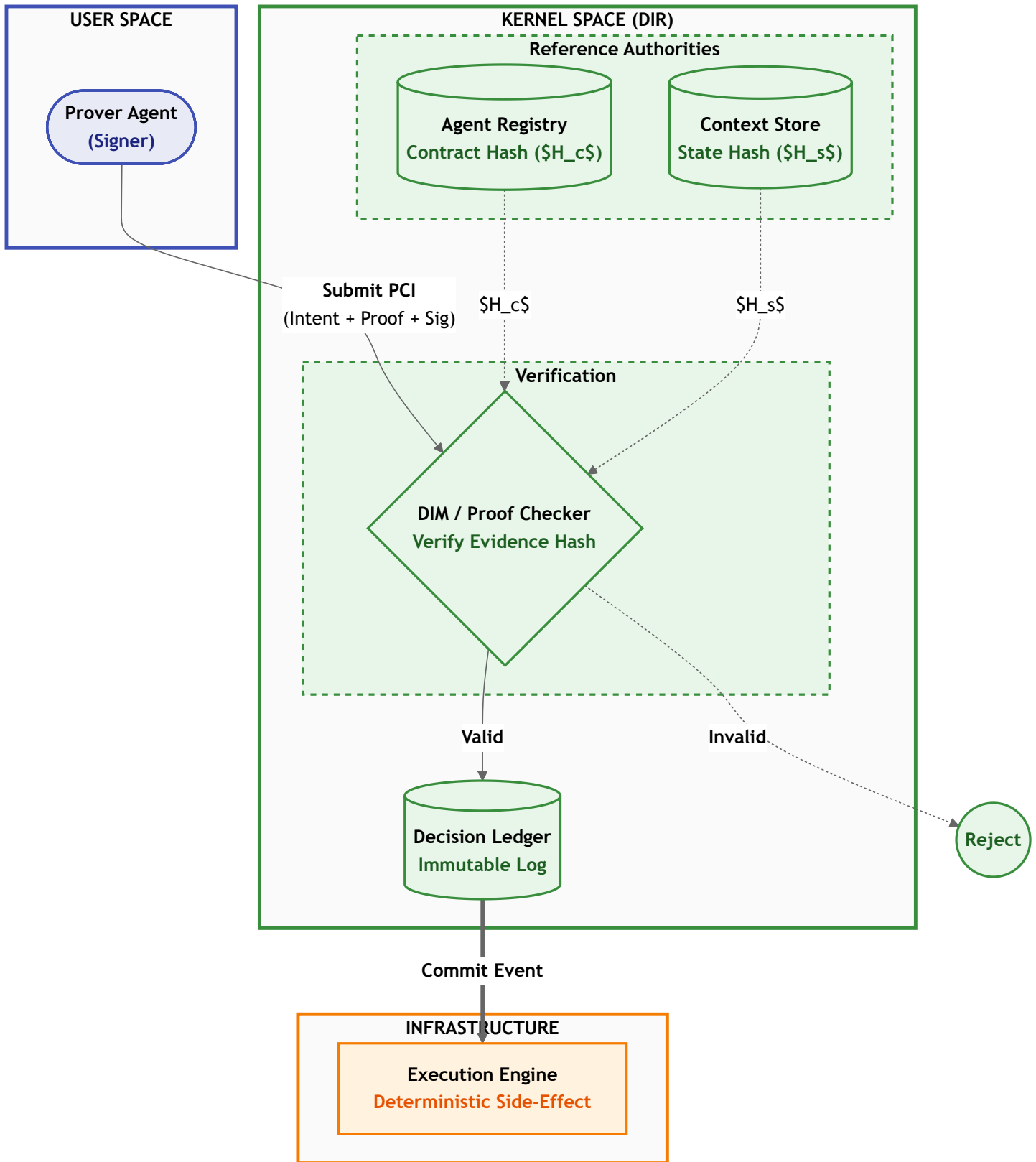
4.4 Concept Refinement: PCI vs. Decision Atom

Proof-Carrying Intent (PCI) is an evolution of the "Decision Atom" from SDS.

- **Decision Atom:** Contains **What** (Action) and **Context** (Snapshot). Guaranteed to be *syntactically* valid by SDS.
- **Proof-Carrying Intent:** Contains **What**, **Why** (Semantic Rationale), and **Proof** (Deterministic evidence). Guaranteed to be *formally* compliant against the Ledger's history.

While SDS ensures the agent stays within *syntactic* bounds (e.g., "produce valid JSON"), DL+PCI ensures **formal compliance** (e.g., "this transfer is valid because the balance was X at block Y, and policy Z allows transfers under limit L").

Topology C: Component Architecture & Proof Flow



5. Comparative Analysis: The Decision Matrix

The choice between EOAM, SDS, and DL+PCI is a strategic decision based on the trade-off between **Coordination**, **Velocity**, and **Integrity**.

Feature	Event-Oriented Agent Mesh (EOAM)	Sovereign Decision Stream (SDS)	Decision Ledger (DL+PCI)
Primary Goal	Coordinated Strategic Reasoning	Atomic execution velocity	Formal Verification / Absolute Integrity
Topology Type	Decentralized Choreography (Many-to-Many)	Linear Pipeline (One-to-One)	Append-Only Log (State-to-State)
Safety Mechanism	Post-Generation Validation (DIM checks proposals)	Syntactic Safety (Grammar enforces rules)	Proof-Carrying Decoupling (Safety in artifact)
Concurrency	High (Parallel Reasoning)	Low (Linear Atomic Decision)	Serialized (Ledger Ordering)
Latency	Significant (Priority-Based Preemption)	Minimal (Single Inference Pass)	Medium (Log persistence & proof verification)
Cost	High (Multi-agent activation)	Low (Token-optimized single call)	Medium (Proof generation overhead)
Ideal Case	Strategic Portfolio Rebalancing	Tactical Scalping / Risk Mitigation	Compliance-heavy ops, Settlements

6. Implementation Blueprints

A robust **Decision Intelligence** system can support all topologies simultaneously within the same **Agent Registry**.

6.1 Topology A: Mesh Handler (EOAM)

```
# agents/trader_mesh_agent.py
class TacticalTrader(ResponsibleAgent):
    def register(self):
        # Register for semantic routing
        self.registry.handshake(self.agent_id, contract, topology="EOAM")

    def on_observation(self, event: ObservationEvent):
        # 1. Mesh Logic: Reactive, Parallel
        context = ContextCompiler.assemble(event.snapshot_id)
        explanation = self.llm.explain(context, self.mission)
        policy_intent = self.llm.generate_policy(context, explanation)

        # 2. Emit Proposal to Bus for Preemption/Selection
        proposal = PolicyProposal(
            dfid=event.dfid,
            params=policy_intent,
            constraints={"drift_envelope_bps": 50}, # JIT Drift Parameter
            context_ref=event.snapshot_id
        )
        EventBus.publish(proposal)
```

6.2 Topology B: Stream Handler (SDS)

```
# sds/core_handler.py
from outlines import models, generate
from pydantic import BaseModel, Field

class SDSExecutionConstraints(BaseModel):
    # The DIR-enforced 'Straightjacket'
    max_trade_size: float = Field(le=1000.0) # Absolute DIR Hard Gate
    authorized_instruments: list[str] = ["BTC-USD", "ETH-USD"]

class SDSPolicy(BaseModel):
    action: str # Must match Literal ["BUY", "SELL", "HOLD"]
    amount: float
    rationale: str # The 'Explain' component

def execute_sovereign_stream(dfid: str, context_snapshot: dict):
    # 1. Fetch Mission and Constraints from Registry & DIR
    contract = AgentRegistry.get_contract(context_snapshot.agent_id)

    # 2. Build the 'Decision Atom' Grammar
    # This prevents the LLM from ever suggesting > 1000.0 or unauthorized assets
    grammar = build_pydantic_grammar(SDSPolicy, constraints=contract.limits)

    # 3. Constrained Inference (The SDS Secret Sauce)
    # The model samplers are gated by the DIR-defined grammar
    policy_proposal = generate.json(model, grammar)(
        prompt=f"Mission: {contract.mission}\nContext: {context_snapshot.data}"
    )

    # 4. JIT Verification & Execution (Minimal Latency)
    # Requires hash-binding check against context_snapshot
    return DIR.jit_execute(dfid, policy_proposal, context_snapshot.hash)
```

6.3 Topology C: Ledger Handler (DL+PCI)

```
# sds/ledger_intent_compiler.py
class ProofCarryingIntent(BaseModel):
    dfid: str
    intent: SDSPolicy # Syntactically bound
    proof: dict = {
        "rule_checks": ["RBAC", "LIMITS", "FRESHNESS"],
        "evidence_hash": "sha256_of_state_and_contract"
    }
    signature: str # Cryptographic bond to ROA identity

class ProofChecker:
    def verify(self, pci: ProofCarryingIntent, ledger: DecisionLedger):
        # 1. Cryptographic Verification
        if not self.verify_signature(pci.signature, pci.dfid):
            raise InvalidSignatureError()

        # 2. Logical Verification
        # Does the evidence hash match the current ledger state?
        current_state_hash = ledger.get_state_hash()
        if pci.proof["evidence_hash"] != current_state_hash:
            raise StateDriftError("Proof invalid for current state")

        # 3. Commit
        ledger.append(pci)
        return True
```

6.4 Kernel-User Space Integrity Constraints

To maintain absolute alignment between the Reasoning (Agent) and Execution (Kernel) layers:

- **Deterministic Compensation Menu:** In multi-step decisions, agents MUST NOT generate "reasoning-based compensation" logic. Instead, they must select from a pre-defined, DIR-validated set of actions (e.g., REVERT , CLOSE_ALL , ALERT_HUMAN). This prevents the same reasoning capability that caused the failure from exacerbating it.
- **Lock Normalization:** Agents are NOT responsible for sorting Resource IDs to prevent deadlocks. **Alphabetical lock normalization is a Kernel (Runtime) responsibility**, ensuring that the Reasoning layer remains infrastructure-agnostic.

7. Conclusion

The development of **Decision Intelligence Topologies** marks the maturation of agentic engineering. We are moving beyond the era of treating Large Language Models as meaningful entities in themselves, and towards an era where they are components within rigorous system architectures.

The inclusion of **Topology C (DL+PCI)** completes the triad:

- **EOAM** for breadth and strategy.
- **SDS** for speed and tactics.
- **DL+PCI** for truth and audit.

Sophisticated organizations will employ all three topologies in concert. The Mesh (EOAM) thinks deeply about strategy and risk; the Stream (SDS) executes those strategies with precision; and the Ledger (DL) records the irrevocable truth of every action. In all cases, the future belongs to architectures that prioritize **auditability over creativity** and **constraints over capabilities**.

8. Glossary

- **Priority-Based Preemption (EOAM):** A mechanism where the Runtime selects the "winning" proposal based on the **Agent Registry Priority Matrix** (e.g., Risk > Strategy) rather than just a time window.
- **Constrained Decoding (SDS):** Using grammar-based sampling to physically prevent an LLM from generating non-compliant tokens.
- **Drift Envelope:** A contract-level parameter specifying the maximum allowable deviation (e.g., price slippage, latency) for JIT Verification before a PolicyProposal is rejected.
- **DecisionFlow ID (DFID):** The immutable trace ID linking observation, reasoning, and execution.
- **Decision Intelligence Runtime (DIR):** The deterministic kernel validating all agent actions.
- **Decision Atom (SDS):** A single, context-complete package for atomic execution, hash-bound to a specific snapshot.
- **Event-Oriented Agent Mesh (EOAM):** A decentralized topology for parallel, multi-agent coordination.
- **Responsibility-Oriented Agent (ROA):** The identity layer defining "Who" is reasoning.
- **Sovereign Decision Stream (SDS):** A linear topology for high-velocity, atomic decisions.
- **Wake-up Predicate:** cheap heuristic logic used to suppress signals before invoking expensive agents.

- **Decision Ledger:** An append-only, immutable record serving as the source of truth for all decisions.
- **Proof-Carrying Intent (PCI):** An intent artifact containing action, rationale, and deterministic proof of rule compliance.
- **Proof Checker:** A minimalist verification module that validates PCI proofs without reasoning.
- **Immutable Narrative:** The unchangeable history of decisions and states stored in the Decision Ledger.

Technical Annex: Cryptographic Integrity for Topology C (DL+PCI)

1. Abstract

This technical annex defines the protocol for generating and verifying the **Evidence Hash** and **Proof-Carrying Intents (PCI)** within the **Decision Ledger** topology. As established in the **Decision Intelligence Topologies** whitepaper, Topology C moves the system from "Active Reasoning" to "Formal Verification," where safety is a structural property of the intent artifact itself.

The primary objective is to ensure that every entry in the **Decision Ledger** is cryptographically bound to a specific version of reality (**Context Snapshot**), a specific authority (**Responsibility Contract**), and a deterministic result from the **Proof Checker**.

2. The Proof-Carrying Intent (PCI) Structure

A **Proof-Carrying Intent** is the definitive communication primitive for Topology C. It encapsulates the probabilistic reasoning of the agent within a deterministic, verifiable envelope.

Field	Type	Description
dfid	UUID	The immutable DecisionFlow ID linking the intent to its causal trigger.

Field	Type	Description
intent_payload	JSON	The Syntactically Bound policy generated via SDS Constrained Decoding.
context_ref	Hash	The ContextSnapshotID representing the "frozen reality" used during reasoning.
evidence_hash	Hash	The composite hash proving alignment between state, contract, and rules.
roa_signature	Signature	The cryptographic signature of the agent, verifying identity and authority.

3. Evidence Hash Generation Protocol

The **Evidence Hash** is not a simple checksum; it is a **Merkle-root-style** derivation that ensures any drift in context or unauthorized changes to the mission results in an immediate verification failure.

3.1 Components of the Hash

The hash is derived from three immutable inputs provided by the **Decision Intelligence Runtime (DIR)**:

1. **State Snapshot Hash (H_s)**: The SHA-256 hash of the ContextSnapshotID .
2. **Contract Hash (H_c)**: The SHA-256 hash of the **Responsibility Contract** active in the **Agent Registry** at T_{start} .
3. **Rule-Set Hash (H_r)**: The hash of the deterministic validation rules (Hard Gates) applied by the DIM.

3.2 Derivation Formula

The evidence_hash ($H_{evidence}$) is calculated as follows:

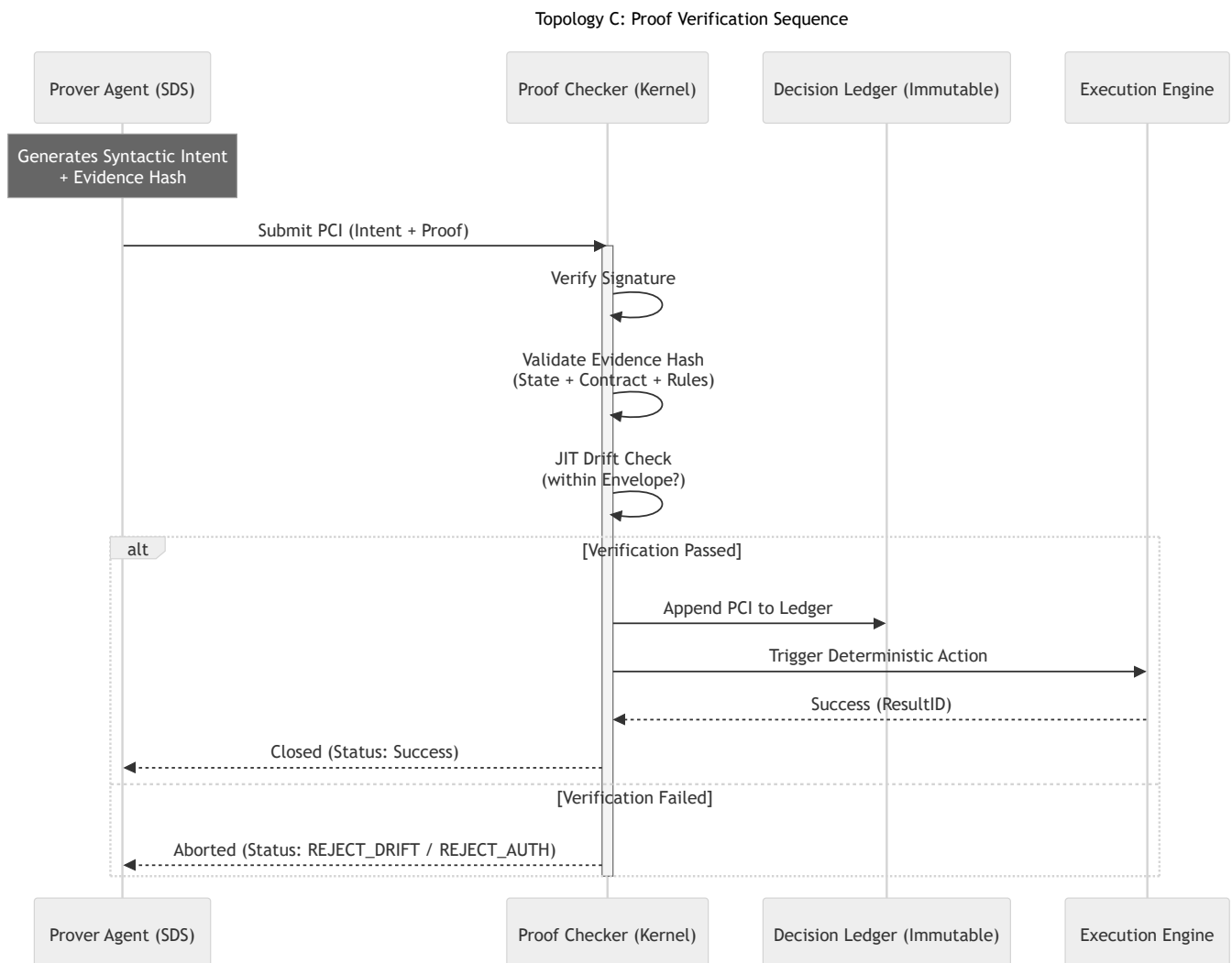
$$H_{evidence} = \text{SHA256}(\text{DFID} \parallel H_s \parallel H_c \parallel H_r)$$

This ensures that the proof is unique to the specific **DecisionFlow** and cannot be replayed in a different context or by an agent with a different contract.

4. The Proof Checker Workflow

In Topology C, the **Decision Integrity Module (DIM)** functions as a minimalist **Proof Checker**. It executes the following deterministic steps before committing the intent to the **Decision Ledger**:

1. **Identity Attestation:** Verifies that the `roa_signature` matches the public key registered in the **Agent Registry**.
2. **Context Binding:** Retrieves the current system state and verifies that the `context_ref` in the PCI matches the `ContextSnapshotID` generated at the start of the flow.
3. **JIT Drift Verification:** Performs a final check to ensure the live environment state is within the **Drift Envelope** defined in the contract.
4. **Hash Validation:** Re-calculates the `evidence_hash` using the Registry's authoritative data and compares it against the hash provided in the PCI.
5. **Commitment:** If all checks pass, the PCI is appended to the **Decision Ledger**, and the **Execution Engine** is triggered.



5. Resilience: Handling State Disruption

Topology C utilizes the **Decision Ledger** to manage failures without relying on probabilistic agent reasoning.

- **Audit Trait:** Because every intent is a PCI, the Ledger provides a "Zero-Trust" audit trail. If a system failure occurs, the Ledger allows for a **Deterministic Replay** to identify exactly which proof failed.
- **Immutable Compensation:** If an execution fails after the Ledger commitment (e.g., Infrastructure error), the system triggers a **Deterministic Compensation Action** from a pre-defined menu (e.g., `REVERT_STATE`), rather than asking the agent to re-reason about the failure.
- **Registry Outage:** During a Registry outage, the Proof Checker relies on **Local Manifest Caching**. However, new PCIs requiring authority updates are rejected to prevent **Stale Authority Exploitation**.

6. Conclusion

The **Evidence Hash** and **Proof-Carrying Intent** represent the most rigorous layer of the **Decision Intelligence Runtime**. By treating safety as a cryptographic requirement of the artifact, Topology C ensures that even the most advanced Large Language Models remain "Provers" of intent rather than "Owners" of execution. This annex provides the technical substrate for a system where truth is found in the **Ledger**, and agents are held to an immutable standard of formal compliance.